

Discrete Optimization

Two phase algorithms for the bi-objective assignment problem

Anthony Przybylski ^{a,*}, Xavier Gandibleux ^a, Matthias Ehrgott ^{a,b}^a *LINA – Laboratoire d’Informatique de Nantes Atlantique, FRE CNRS 2729, Université de Nantes,
2 Rue de la Houssinière BP 92208, 44322 Nantes Cedex 03, France*^b *Department of Engineering Science, The University of Auckland, Private Bag 92019, 70 Symonds Street,
Auckland Mail Center, Auckland 1142, New Zealand*

Received 14 December 2005; accepted 22 December 2006

Available online 13 February 2007

Abstract

In this paper, we present several algorithms for the bi-objective assignment problem. The algorithms are based on the two phase method, which is a general technique to solve multi-objective combinatorial optimisation (MOCO) problems.

We give a description of the original two phase method for the bi-objective assignment problem, including an implementation of the variable fixing strategy of the original method. We propose several enhancements for the second phase, i.e., improved upper bounds and a combination of the two phase method with a population based heuristic using path relinking to improve computational performance. Finally, we describe a new technique for the second phase with a ranking approach, which outperforms all other tested algorithms.

All of the algorithms have been tested on instances of varying size and range of objective function coefficients. We discuss the results obtained and explain our observations based on the distribution of objective function values.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Multi-objective optimisation; Integer programming; Assignment problem; Two phase method; Heuristic; Path relinking; Efficient solution; Ranking algorithm; Combinatorial optimization; Metaheuristics

1. The bi-objective assignment problem

In this paper, we investigate the exact solution of the bi-objective assignment problem. In Section 1, we introduce the problem, general definitions and a classification of efficient solutions. Our investigation is within the framework of the two phase method, originally proposed for the bi-objective assignment problem in [Ulungu and Teghem \(1995\)](#). Section 2 is mainly devoted to the description of that solution method, which we refer to as “the original two phase method”. In Section 3, we present improvements of the original method and new algorithms for the second phase. These include the incorporation of a population based heuristic

* Corresponding author. Tel.: +33 0 251125822.

E-mail addresses: Anthony.Przybylski@univ-nantes.fr (A. Przybylski), Xavier.Gandibleux@univ-nantes.fr (X. Gandibleux), Matthias.Ehrgott@univ-nantes.fr, m.ehrgott@auckland.ac.nz (M. Ehrgott).

using path-relinking to obtain good starting solutions and the use of a ranking algorithm. All proposed algorithms are evaluated on a large set of numerical instances. Numerical results are reported and discussed in Section 4. The paper is concluded with a discussion on the distribution of objective values.

1.1. Problem formulation and characteristics

The single objective assignment problem (AP) is an integer programming problem that can be solved as a linear program due to total unimodularity of the constraint matrix. Efficient algorithms to solve it, e.g., the Hungarian method or the successive shortest paths method (Papadimitriou and Steiglitz, 1982; Ahuja et al., 1993) are well known.

In this paper, we consider the assignment problem with two objectives (BAP). It can be formulated as follows:

$$\begin{aligned} \min \quad & z_k(x) = \sum_{i=1}^n \sum_{j=1}^n c_{ij}^k x_{ij}, \quad k = 1, 2, \\ & \sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, n \\ & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, n \\ & x_{ij} \in \{0, 1\}, \quad i, j = 1, \dots, n, \end{aligned} \tag{BAP}$$

where all objective function coefficients c_{ij}^k are non-negative integers and $x = (x_{11}, \dots, x_{nn})$ is the matrix of decision variables.

Let X denote the set of all feasible solutions of (BAP). $X \subset \{0, 1\}^{n^2} \subset \mathbb{R}^{n^2}$ is also called the feasible set in decision space. $Z = \{z(x) : x \in X\} \subset \mathbb{N}^2 \subset \mathbb{R}^2$ is called the feasible set in objective space.

In bi-objective optimisation, there is in general no feasible solution which minimises both objectives simultaneously.

Definition 1. A feasible solution $x^* \in X$ is called *efficient* if there does not exist any other feasible solution $x \in X$ such that $z_k(x) \leq z_k(x^*)$, $k = 1, 2$, with at least one strict inequality. $z(x^*)$ is then called a *non-dominated point*. The set of efficient solutions is denoted by X_E and the image of X_E in Z is called the *non-dominated frontier* Z_N . If $x, x' \in X$ are such that $z_k(x) \leq z_k(x')$, $k = 1, 2$, and $z(x) \neq z(x')$ we say that x dominates x' ($z(x)$ dominates $z(x')$).

The set of efficient solutions is partitioned in two subsets as follows.

- *Supported* efficient solutions are optimal solutions of a weighted sum single objective problem:

$$\min \{ \lambda_1 z_1(x) + \lambda_2 z_2(x) : x \in X \} \tag{BAP}_\lambda$$

for some $\lambda_1, \lambda_2 > 0$. All supported non-dominated points are located on the “lower-left boundary” of the convex hull of Z ($\text{conv } Z$), i.e., they are non-dominated points of $(\text{conv } Z) + \mathbb{R}_+^2$. By varying λ all supported solutions can be found. We use the notations X_{SE} and Z_{SN} , respectively, to denote supported efficient solutions and supported non-dominated points.

- *Non-supported* efficient solutions are efficient solutions that are not optimal solutions of $(\text{BAP})_\lambda$ for any λ with $\lambda_1, \lambda_2 > 0$. Non-supported non-dominated points are located in the interior of $(\text{conv } Z) + \mathbb{R}_+^2$. No theoretical characterisation which leads to an efficient computation of the non-supported efficient solutions is known. The sets of non-supported efficient solutions and non-supported non-dominated points are denoted X_{NE} and Z_{NN} , respectively.

In addition we can distinguish two classes of supported efficient solutions.

- Supported efficient solutions x are those whose objective vectors $z(x)$ are located on the vertex set of $\text{conv } Z$. We call these extremal supported efficient solutions X_{SE1} and let $Z_{\text{SN1}} = z(X_{\text{SE1}})$, the extremal supported non-dominated points.
- The set of those $x \in X_{\text{SE}}$ for which $z(x)$ is not located on the vertex set of $\text{conv } Z$. This set is denoted by X_{SE2} and its image by Z_{SN2} .

Both X_{SE1} and X_{SE2} can be obtained by solving (BAP_λ) . Nevertheless, the computation of the latter is generally more difficult than that of the former, because for all efficient solutions x with $z(x)$ on an edge of the convex hull of Z , the weight vector λ for which x minimises (BAP_λ) is the same. Therefore, finding X_{SE2} requires an enumeration of *all* optimal solutions of (BAP_λ) (cf. Fig. 1).

All efficient solutions of multi-objective linear programming problems are supported (see, e.g., Steuer (1985)). Because of total unimodularity of the constraint matrix of (BAP) , one might therefore think that non-supported efficient solutions of the (BAP) do not exist. But this is not true, as Ulungu and Teghem (1995) have pointed out and as can be seen, e.g., in Fig. 2. Thus, the bi-objective assignment problem must be considered as an integer programming problem. Moreover, the following theorem holds.

Theorem 1. *The assignment problem with two objectives is $\mathcal{NP}$ -complete, $\#\mathcal{P}$ -complete, and intractable.*

Here, intractability means that an exponential number of efficient solutions (and non-dominated points) may exist. For a proof see Serafini (1986) for $\mathcal{NP}$ -completeness and Neumayer (1994) for $\#\mathcal{P}$ -completeness and intractability or Ehrgott (2005) for a concise proof of all statements.

1.2. Classification of efficient solutions

For the development of algorithms for the bi-objective assignment problem the following classification of X_{E} into subsets is important.

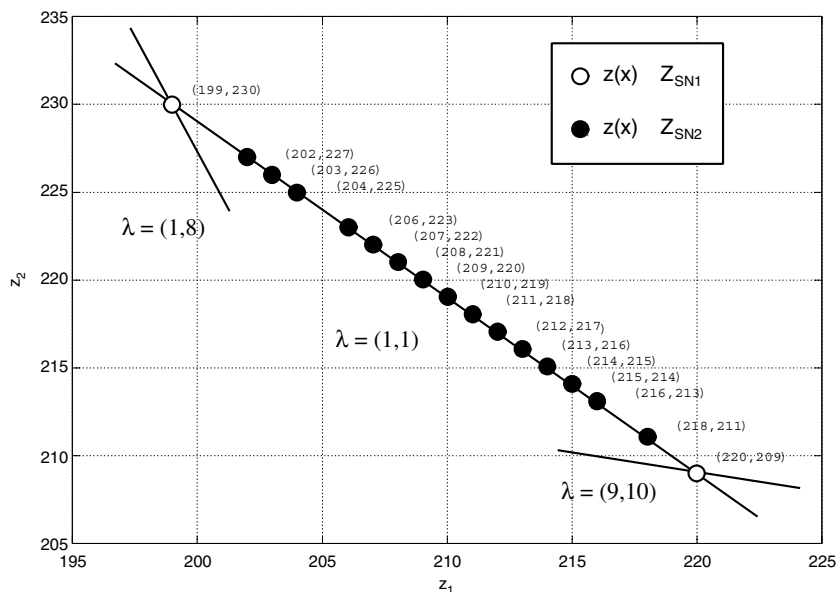


Fig. 1. X_{SE2} : Supported efficient solutions with $z(x)$ on an edge of the convex hull of Z (instance 2AP100–1A20, see Section 4).

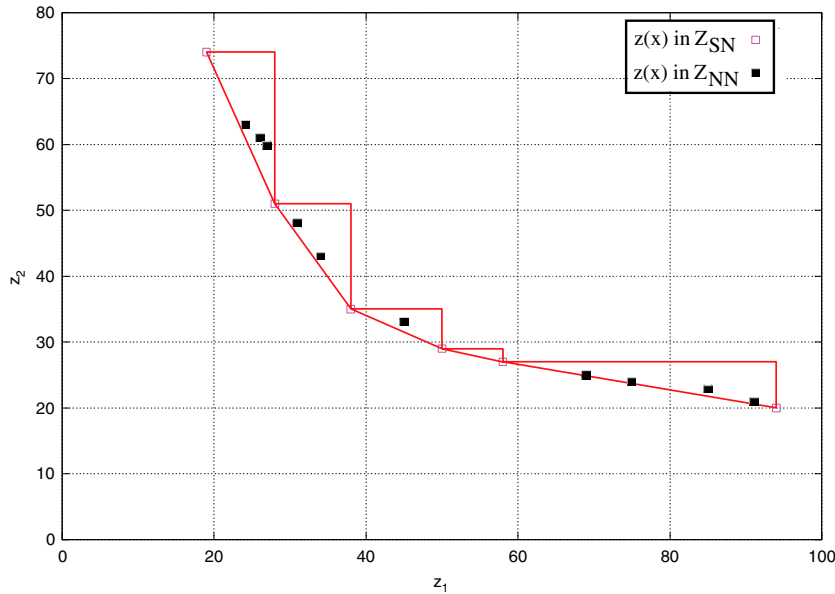


Fig. 2. The exploration space is reduced to the interior of the triangles (instance 2AP10–1A20, see Section 4).

Definition 2

1. (Hansen, 1979) Feasible solutions $x, x' \in X$ are said to be *equivalent* if $z(x) = z(x')$.
2. (Hansen, 1979) A *complete set* X_E is a set of efficient solutions such that all $x \in X \setminus X_E$ are either dominated by or equivalent to at least one $x \in X_E$. I.e., for each non-dominated point $z \in Z_N$ there exists at least one $x \in X_E$ such that $z(x) = z$.
 - (Hansen, 1979) A *minimal complete set* X_{Em} is a complete set without equivalent solutions. Any complete set contains a minimal complete set.
 - The *maximal complete set* X_{EM} is the complete set including all equivalent solutions, i.e. all $x \in X \setminus X_{Em}$ are dominated.

As indicated by intractability of (BAP) the number of equivalent efficient solutions can be large. Indeed, it is known that for some instances of the (BAP) all feasible solutions are efficient. In that case, it is impossible to design an efficient algorithm for computing the maximal complete set X_{EM} . This is the motivation for Hansen's introduction of the notion of minimal complete set X_{Em} .

Supported and non-supported efficient solutions can also be classified using the definitions of Hansen. Thus, we can talk about

- Complete sets of supported and non-supported efficient solutions.
- Minimal and maximal complete sets of supported and non-supported efficient solutions X_{SEm} , X_{NEm} , X_{SEM} , and X_{NEM} .
- Minimal and maximal sets of extremal and non-extremal supported efficient solutions X_{SE1m} , X_{SE2m} , X_{SE1M} and X_{SE2M} .

Published papers are sometimes vague about the capabilities of proposed algorithms. Some authors claim that their algorithm can enumerate “all” efficient solutions, but do not mention whether that includes equivalent solutions, or even solutions in X_{SE2} . To clearly state the characteristic of existing algorithms, it is important to give the class of efficient solutions that an algorithm computes.

1.3. Literature

According to Ehrgott and Gandibleux (2002), early papers on multiple objective assignment problems only deal with supported efficient solutions, using convex combinations of objective functions or goal programming. Some algorithms to determine a complete set X_E have been proposed: Malhotra et al. (1982) assume that the efficient solutions are connected by simplex pivots. Ulungu (1993) shows that this assumption is false. Ulungu and Teghem (1995) propose an operational procedure for the BAP derived from the two phase method. However, this procedure turns out to be very time consuming (see Tuytens et al. (2000)). Consequently, approximation methods have been proposed for computing sub-optimal solutions within reasonable computing time. Two methods have been proposed for the assignment problem: an extension of simulated annealing to deal with multiple objectives, called the MOSA method (Ulungu, 1993), and a population-based heuristic using path relinking (Gandibleux et al., in press).

2. The original two phase method

2.1. Principle and literature

The two phase method is a general framework for solving MOCO problems (Ulungu and Teghem, 1995), although it has never been applied for problems with more than two objectives. The main idea is to use efficient algorithms for the single objective problem to compute efficient solutions. As efficient algorithms for single objective problems are problem specific it is necessary to preserve the constraint structure of the problem throughout the solution procedure. It is therefore, for example, not possible to add constraints on objective function values (as is done in multi-objective integer programming methods, e.g., Mavrotas and Diakoulaki (1998), Ehrgott and Ryan (2003)) to search for efficient solutions.

In Phase 1, supported efficient solutions are computed. It is based on Geoffrion's theorem (Geoffrion, 1968) stating that an optimal solution of (BAP_λ) with positive weights is efficient.

In Phase 2, information from supported efficient solutions identified by Phase 1 is used to reduce the search space for computing the non-supported efficient solutions. In implementations, Phase 2 is generally enumerative.

In the bi-objective case, Phase 2 explores the triangles defined by two consecutive supported solutions (Fig. 2) in objective space. To search for non-supported efficient solutions in an effective manner, lower bounds, upper bounds, reduced costs, etc. are usually employed.

The method has been applied to a number of problems other than (BAP), e.g., network flow (Lee and Pulat, 1993; Sedeño-Noda and González-Martín, 2001), knapsack (Ulungu, 1993; Visée et al., 1998), and spanning tree (Ramos et al., 1998). It has first been proposed for the bi-objective assignment problem by Ulungu and Teghem (1995). Some experimental results have been published by Tuytens et al. (2000). But the results obtained by Degoutin and Gandibleux (2002) using an MIP solver have shown more efficient solutions for the same instances. This put the validity of the implementation in question.

In the rest of this section, we describe the original method where we incorporate some technical details in the algorithm to speed it up.

2.2. Phase 1: Determination of supported efficient solutions

The Phase 1 algorithm (see Algorithms 1 and 2) described in this section is used unchanged in all two phase algorithms described in the rest of the paper.

Phase 1 is a variation of the dichotomic scheme proposed by Aneja and Nair (1979). It determines the set X_{SE_M} . Let S denote a set of efficient solutions found by the algorithm. $S \leftarrow \{x^1, x^2\}$ is initialised with the two lexicographic optimal solutions corresponding to $\text{lexmin}_{x \in X}(z_1(x), z_2(x))$ and $\text{lexmin}_{x \in X}(z_2(x), z_1(x))$, respectively.

Algorithm 1**Procedure BAP_Phase1**

Parameters $\downarrow$: The cost matrices C^1 and C^2
Parameters $\uparrow$: The set X_{SEM}

- | Compute the lexicographically optimal solution x^1 for (z_1, z_2)
 - | Consider (BAP_λ) , the single objective optimisation problem with the cost matrix
 - | $C^{1'} = [\lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2]$ with $\lambda_1 = z_2(x^{1'}) + 1$ and $\lambda_2 = 1$
 - solveAP ($C^1 \downarrow, x^{1'} \uparrow, \overline{C} \uparrow$)
 - solveAP ($C^{1'} \downarrow, x^1 \uparrow, \overline{C} \uparrow$)
 - | Compute the lexicographically optimal solution x^2 for (z_2, z_1)
 - | Consider (BAP_λ) , the single objective optimisation problem with the cost matrix
 - | $C^{2'} = [\lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2]$ with $\lambda_1 = 1$ and $\lambda_2 = z_1(x^{2'}) + 1$
 - solveAP ($C^2 \downarrow, x^{2'} \uparrow, \overline{C} \uparrow$)
 - solveAP ($C^{2'} \downarrow, x^2 \uparrow, \overline{C} \uparrow$)
 - | Compute X_{SEM}
 - $S \leftarrow \{x^1, x^2\}$; solveRecursion($x^1 \downarrow, x^2 \downarrow, S \downarrow$); $X_{SEM} \leftarrow S$
-

Comment. In the algorithms, the symbols $\downarrow$, $\uparrow$ and $\updownarrow$ specify the transmission mode of a parameter to a procedure; they correspond respectively to the mode IN, OUT and IN OUT. The symbol - | marks the beginning of a comment line.

2.2.1. Finding lexicographic extreme efficient solutions

To determine x^1 and x^2 , we first compute one optimal solution (using procedure solveAP in Algorithms 1 and 2) for each of the two single objective problems ($x^{1'}$ for $\min_{x \in X} z_1(x)$ and $x^{2'}$ for $\min_{x \in X} z_2(x)$) by solving $(BAP_{(1,0)})$ and $(BAP_{(0,1)})$. These solutions may be dominated.

While x^1 and x^2 can be found by solving $\min\{z_2(x) : x \in X, z_1(x) \leq z_1(x^{1'})\}$ and $\min\{z_1(x) : x \in X, z_2(x) \leq z_2(x^{2'})\}$, respectively, or by enumerating all optimal solutions of $(BAP_{(1,0)})$ and $(BAP_{(0,1)})$, these techniques might be time consuming in practice, because a single objective assignment problem may have an exponential number of optimal solutions and (BAP_λ) with a constraint (derived from an objective function) is $\mathcal{NP}$ -hard (Mazzola and Neebe, 1986).

To compute x^1 and x^2 , we propose the following procedure, which we describe here for the first objective. It is possible to find x^1 by solving an additional (BAP_λ) with the weight vector $(1, 0)$ slightly modified. Because $c_{ij}^k \in \mathbb{N}$ we know $Z \subset \mathbb{N}^2$. If we use weights defined by the normal to the line through $(z_1(x^{1'}), z_2(x^{1'}))$ and $(z_1(x^{1'}) + 1, -1)$, i.e., $\lambda_1 = z_2(x^{1'}) + 1$ and $\lambda_2 = 1$, the optimal solutions of the corresponding (BAP_λ) are optimal solutions of $\text{lexmin}_{x \in X}(z_1(x), z_2(x))$. Note that we assume all c_{ij}^k to be integer.

x^2 is found the same way by using $x^{2'}$ and solving problem (BAP_λ) with weights $\lambda_1 = 1$ and $\lambda_2 = z_1(x^{2'}) + 1$.

Once the lexicographic optimal solutions are found the dichotomic search begins.

2.2.2. Finding a complete set of supported solutions

During the dichotomic search, the solutions of S are sorted by increasing z_1 values. Two non-equivalent consecutive solutions x^r and x^s according to this order, i.e., $z_1(x^r) < z_1(x^s)$ and $z_2(x^r) > z_2(x^s)$, are considered. A weighted sum problem (BAP_λ) with $\lambda_1 = z_2(x^r) - z_2(x^s)$ and $\lambda_2 = z_1(x^s) - z_1(x^r)$ is solved, and all optimal solutions are enumerated. In our work, we use the algorithm of Fukuda and Matsui (1992) (using procedure enumerate in Algorithm 2). The algorithm is initialised with $x^r = x^1$ and $x^s = x^2$.

Algorithm 2**Procedure solveRecursion**Parameters $\downarrow$: x^r, x^s Parameters $\updownarrow$: S

- | Compute the set R of optimal solutions x of (BAP_λ) :

```

-|  $\min\{\lambda_1 z_1(x) + \lambda_2 z_2(x) : x \in X\}$ 
-| where  $\lambda_1 = z_2(x^r) - z_2(x^s)$ , and  $\lambda_2 = z_1(x^s) - z_1(x^r)$ .
-|  $c_{ij}^\lambda = \lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2$ ,  $i = 1, \dots, n$ ;  $j = 1, \dots, n$ 
solveAP ( $C^\lambda \downarrow, x \uparrow, \overline{C^\lambda} \uparrow$ ); enumerate ( $\overline{C^\lambda} \downarrow, x \downarrow, R \uparrow$ );  $R \leftarrow R \cup \{x\}$ 
 $S \leftarrow S \cup R$ 
if  $\{z^\lambda(x) : x \in R\} \cap \overline{z(x^r)z(x^s)} = \emptyset$  then
-| Case (a)
-| Let  $x^{t1}$  and  $x^{t2}$ , the solutions of  $R$  with respectively the min. and max. value on  $z_1$ 
  solveRecursion ( $x^r \downarrow, x^{t1} \downarrow, S \uparrow$ ); solveRecursion ( $x^{t2} \downarrow, x^s \downarrow, S \uparrow$ )
end if
-| if  $\{z^\lambda(x) : x \in R\} \subset \overline{z(x^r)z(x^s)}$  then
-| Case (b) : nothing to do

```

Let $R = \{x^t : t \in T\}$ be all optimal solutions of (BAP_λ) , where T is an index set such that $|T|$ is the number of optimal solutions of (BAP_λ) . For two points y^r and y^s in $\mathbb{R}^2$, we denote by $\overline{y^r y^s}$ the line segment joining the points y^r and y^s . Two cases can occur:

- (a) $R \cap \overline{z(x^r)z(x^s)} = \emptyset$, see Fig. 3. All solutions x^t are new supported efficient solutions and are added to S . Then the minimum and maximum of $z_1(x)$ in $\{x^t : t \in T\}$ are computed. Let x^{t1} and x^{t2} be the solutions where the minimum and maximum are attained. Two new weighted sum problems are considered, one defined by x^r and x^{t1} and one defined by x^{t2} and x^s (using procedure solveRecursion in Algorithms 1 and 2). It is not necessary to consider a scalarised problem defined by two solutions of R because the weight vector λ will be the same as that defined by x^r and x^s and therefore no new solutions will be found.
- (b) $R \subset \overline{z(x^r)z(x^s)}$, see Fig. 4. All solutions x^t are supported efficient solutions and the new solutions are added to S , but no new scalarised problem is generated.

Phase 1 stops if no new weighted sum problems (BAP_λ) have to be solved with $S = X_{\text{SEM}}$. Note that without using an enumerative algorithm, we will in general only find a minimal complete set X_{SE1_m} and possibly some other supported efficient solutions. The enumerative algorithm is therefore necessary to be sure to find a set X_{SE2_m} , and of course also X_{SEM} .

2.3. Phase 2: Determination of non-supported efficient solutions

Phase 2 must determine $x \in X$ such that $z(x)$ is in the triangle defined by two consecutive non-dominated supported points $z(x^r)$ and $z(x^s)$ in the objective space (see Fig. 2). In order to limit the exploration in the triangle, lower and upper bounds on objective values of single objective assignment problems have been proposed by Ulungu and Teghem (1995).

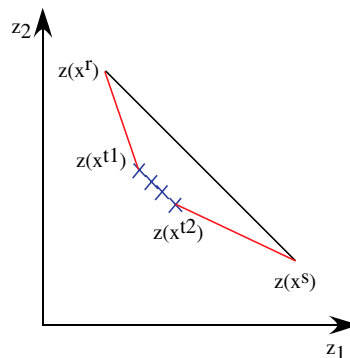


Fig. 3. Phase 1, Case (a).

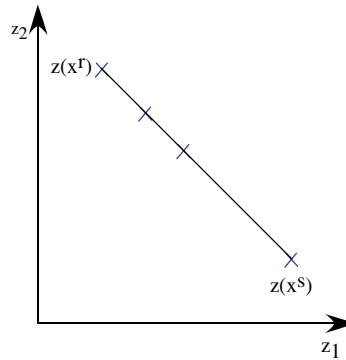


Fig. 4. Phase 1, Case (b).

2.3.1. Lower bounds

Lower bounds avoid the exploration of solutions that cannot be efficient. Let $x \in X$ and let C be the matrix of objective function coefficients of a single objective AP. We are interested in the objective value that results from fixing a variable $x_{ij} = 1$. In the algorithms C will be either C^1, C^2 or $C^\lambda = \lambda_1 C^1 + \lambda_2 C^2$ with $\lambda_1 > 0$ and $\lambda_2 > 0$, or a square submatrix of any of these.

Ulungu and Teghem (1995) have proposed two different lower bounds depending on whether x is optimal for the single objective problem or not. In the following, we denote by $\alpha_1(C, x, (i, j))$ the lower bound on the objective value with matrix C of objective function coefficients, with x an optimal solution for this objective, and with (i, j) the imposed assignment (i.e., x_{ij} is to be fixed to one). By analogy, we denote by $\alpha_2(C, x, (i, j))$ the lower bound on the objective value if x is not an optimal solution for the objective C .

2.3.2. Upper bound of Tuytens et al. (2000)

In Phase 2 algorithms, each triangle defined by two consecutive supported efficient solutions is explored separately. Let x^r and x^s be two consecutive supported efficient solutions and let λ be the weight vector for which both x^r and x^s are optimal solutions of (BAP_λ) . Ulungu and Teghem (1995) and Tuytens et al. (2000) have derived upper bounds on the objective value $\lambda_1 z_1(x) + \lambda_2 z_2(x)$ for efficient solutions x with $z(x)$ in the triangle defined by $z(x^r)$ and $z(x^s)$. Let $\Delta(x^r, x^s)$ denote (the interior of) that triangle.

The feasible points in a band in triangle $\Delta(x^r, x^s)$ have to be enumerated. A band is an area in the triangle between the line segment $z(x^r)z(x^s)$ and a line parallel to that. In the worst case, the parallel line contains point $(z_1(x^s), z_2(x^r))$, i.e., all solutions in the triangle are enumerated. Upper bounds allow to use a line closer to $z(x^r)z(x^s)$.

Tuytens et al. (2000) improve the upper bound of Ulungu and Teghem (1995) by using information from all points already explored to reduce this band. To illustrate this bound (and the ones in Section 3.1), we will use a triangle of the instance 2AP30-1A20 (see Section 4) defined by supported efficient solutions x^r, x^s with $z(x^r) = (82, 63)$ and $z(x^s) = (99, 51)$ as an example. The non-dominated points in this triangle are $(86, 61)$, $(88, 60)$, $(91, 59)$, $(92, 58)$, $(93, 56)$, $(95, 55)$, $(97, 54)$, $(98, 52)$. These points are shown, e.g., in Fig. 5.

In Fig. 5, the eight non-dominated points and the area they dominate are shown. We can see that parallel to the line segment $z(x^r)z(x^s)$ there is a line, so that all points of the triangle dominated by a point on that line are also dominated by one of the non-dominated points. Because this line is parallel to $z(x^r)z(x^s)$ only one point is necessary to define it. If we sort the non-dominated points (including $z(x^r)$ and $z(x^s)$) by increasing z_1 values, this line passes through one of the local nadir points defined by two consecutive solutions. The local nadir point of solutions x', x'' with $z_1(x') < z_1(x'')$ and $z_2(x') > z_2(x'')$ is $(z_1(x''), z_2(x'))$.

The algorithm works with a set of potentially efficient solutions X_{PE} identified so far. Let Z_{PN} the corresponding set of potentially non-dominated points and let $\{x^i : 0 \leq i \leq q\}$ be X_{PE} sorted by increasing z_1 values. Let $\gamma = \max_{i=1}^q \{\lambda_1 z_1(x^i) + \lambda_2 z_2(x^{i-1})\}$. Thus an upper bound on any non-dominated point in the triangle is

$$\beta_0 := \max \{ \gamma, \lambda_1 z_1(x^0) + \lambda_2 z_2(x^r), \lambda_1 z_1(x^s) + \lambda_2 z_2(x^q) \}.$$

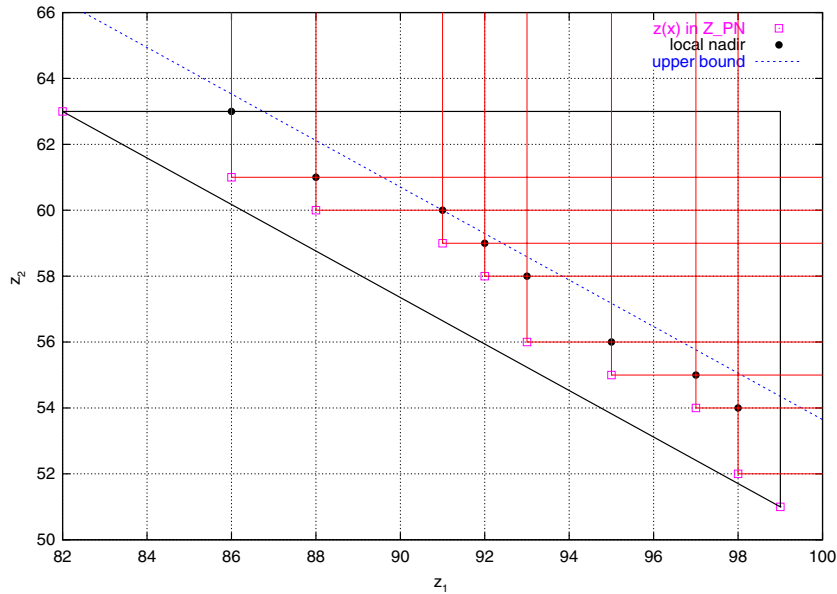


Fig. 5. The bound of Tuytens et al. (2000) defines this dotted line which is located in the dominated area.

In Fig. 5, the maximum is found for the local nadir point (91, 60) with value 2112 (an increase of 57 from x^r and x^s).

As all the solutions $x \in X, z(x) \in \Delta(x^r, x^s)$ with $\lambda_1 z_1(x) + \lambda_2 z_2(x) \geq \beta_0$ are dominated, enumeration of all $x \in X$ with $z(x)$ between the two lines yields all non-supported efficient solutions with $z(x)$ in the triangle, including the equivalent ones. After exploring all triangles the maximal complete set X_{EM} is obtained.

2.3.3. Description of the original method

Let x^r and x^s be two consecutive supported efficient solutions that are not equivalent and let λ be such that both x^r and x^s are optimal solutions of (BAP_λ) . Phase 2 finds feasible solutions x with $z_1(x^r) < z_1(x) < z_1(x^s)$ and $z_2(x^r) > z_2(x) > z_2(x^s)$.

Obviously, all such solutions are in the triangle $\Delta(x^r, x^s)$. Ulungu and Teghem (1995) propose an enumerative exploration method to search for these solutions. The idea is to find non-supported efficient solutions by imposing assignments, i.e., fixing variables $x_{ij} = 1$ in (BAP_λ) , and solving (reoptimising) the reduced assignment problem that results. In other words, Phase 2 looks for non-optimal feasible solutions x of (BAP_λ) starting from x^r and x^s and modifying these solutions.

Ulungu and Teghem (1995) propose to use the list of assignments

$$L = \{(i, j) : \bar{c}_{ij}^\lambda > 0\},$$

where $\bar{C}^\lambda$ denotes the reduced cost matrix obtained by the Hungarian method for the problem (BAP_λ) for which x^r and x^s are optimal. By imposing an assignment of L , solutions with corresponding points located above the straight line joining $z(x^r)$ and $z(x^s)$ are obtained. But it is possible to delete assignments from L by testing if imposing them yields only points outside $\Delta(x^r, x^s)$. To do this, lower bounds of Ulungu and Teghem (1995) are used. There are three tests:

Test 1: Lower bound l_λ on z_λ compared with $z^\lambda(x^r) = z^\lambda(x^s)$.

l_λ is given by $\max\{\alpha_1(C^\lambda, x^r, (i, j)), \alpha_1(C^\lambda, x^s, (i, j))\}$. The assignment can be deleted from L , if l_λ shows that a solution containing this assignment ($x_{ij} = 1$) will be beyond the line parallel to $\overline{z(x^r)z(x^s)}$ containing the point $z = (z_1(x^s), z_2(x^r))$. The increase of z^λ from the supported efficient solutions to z is $\lambda_1 z_1(x^s) + \lambda_2 z_2(x^r) - \lambda_1 z_1(x^r) - \lambda_2 z_2(x^r) = \lambda_1 \lambda_2$.

Consequently, the assignment can be deleted if

$$l_{\lambda} \geq z_{\lambda}(x^r) + \lambda_1 \lambda_2.$$

Since we assume, like Tuytens et al. (2000), the objective coefficients to be integer, we propose an improvement to this test. The assignment can be deleted from L , if l_{λ} shows that a solution containing this assignment will be strictly beyond the line parallel to $z(x^r)z(x^s)$ containing the point $z = (z_1(x^s) - 1, z_2(x^r) - 1)$. Consequently, the assignment can be deleted if

$$l_{\lambda} > z^{\lambda}(x^r) + \lambda_1 \lambda_2 - \lambda_1 - \lambda_2.$$

Test 2: Lower bound l_1 on z_1 compared with $z_1(x^s)$.

l_1 is given by either $\alpha_1(C^1, x^r, (i, j))$ or $\alpha_2(C^2, x^r, (i, j))$ depending on whether x^r is an optimal solution of $(BAP_{(1,0)})$ or not. The assignment can be deleted from L , if l_1 shows that the solution will be to the right of a vertical line through $z(x^s)$, i.e., if

$$l_1 \geq z_1(x^s).$$

Test 3: Lower bound l_2 on z_2 compared with $z_2(x^r)$ (with the reduced costs obtained by the Hungarian method for z_1 if $x^s = x^2$).

l_2 is given by either $\alpha_1(C^2, x^r, (i, j))$ or $\alpha_2(C^2, x^r, (i, j))$ depending on whether x^s is an optimal solution of $(BAP_{(0,1)})$ or not. The assignment can be deleted if l_2 shows that the solution will be above a horizontal line through $z(x^r)$, i.e., if

$$l_2 \geq z_2(x^r).$$

The reduced list of assignments L can be used for the exploration of $\Delta(x^r, x^s)$. During this search, the feasible solutions generated are stored in a list of potentially efficient solutions X_{PE} . Each new solution x is compared to the solutions of X_{PE} , and X_{PE} is updated if necessary. At termination of the search, i.e., if all solutions $x \in \Delta(x^r, x^s)$ not yet enumerated are guaranteed to satisfy $\lambda_1 z_1(x) + \lambda_2 z_2(x) \geq \beta_0$ (upper bound in Section 2.3.2), X_{PE} is a complete set of non-supported efficient solutions in the triangle.

However, by individually imposing each of the assignments of L , we only partially explore the triangle. Ulungu and Teghem (1995) end the description of their method with this remark (the emphasis is ours and “the above inequality” refers to the upper bound of Ulungu and Teghem (1995)).

In fact by setting one assignment $(i, j) \in L$ the generated solution can be outside of the triangle $\Delta(x^r, x^s)$. Nevertheless, by setting simultaneously (i, j) and *some other assignment* of L we can generate a solution x'' with greater value $z^{\lambda}(x'')$ and yet located inside of the triangle $\Delta(x^r, x^s)$, one must continue the optimisation process of (BAP_{λ}) , by setting in addition of (i, j) , other assignment of L until we find either a solution inside the triangle or the above inequality is verified.

With this only remark, all the assignments of the list L must be successively examined but no particular method to set several assignments simultaneously is described in any further detail in Ulungu and Teghem (1995) or Tuytens et al. (2000). The authors of the latter paper just emphasise that as in Phase 1, it is necessary to obtain all optimal solutions of each considered reduced problem in Phase 2 and that this is a major difficulty, as the number of enumerated solutions can be large. In the following section we will give a full description of the variable fixing strategy.

2.4. A full description of the variable fixing strategy

2.4.1. Imposing and forbidding assignments of L

The main goal of this subsection is to show how finding the same solutions repeatedly by imposing combinations of assignments can be avoided.

We will call a set $\{(i_k, j_k) : k \in K\}$ *compatible assignments* if imposing $x_{i_k j_k} = 1$ for all $k \in K$ does not violate the constraints of (BAP) . To avoid double indexing we will write x_k below instead of $x_{i_k j_k}$. Before imposing an additional assignment, we will verify that it is compatible with the ones that have already been imposed.

Suppose $L = \{x_1, x_2, x_3\}$ and that all assignments are compatible. We propose a depth-first search. Because imposing $\{x_1, x_2\}$ and $\{x_2, x_1\}$ will yield the same solution, we can suppose that L is sorted and that the indices are in increasing order. With $L = \{x_1, x_2, x_3\}$ the following possible sets of variables can be fixed to 1: $\{x_1\}$, $\{x_1, x_2\}$, $\{x_1, x_2, x_3\}$, $\{x_1, x_3\}$, $\{x_2\}$, $\{x_2, x_3\}$, $\{x_3\}$.

All solutions containing x_1 are found in the four first sets. By imposing $\{x_2\}$, a solution containing x_1 may be found. However, it has already been discovered. This can be avoided by fixing $x_1 = 0$ in the fifth set of variables, i.e., forbidding x_1 . This can be done by replacing the cost of x_1 by a very high number. In the same way, we know that after the three first sets, we have all solutions containing x_1 and x_2 , so we can forbid x_2 in the fourth set of assignments $\{x_1, x_3\}$.

In general, suppose we have a set of variables $S = \{x_{i_1}, x_{i_2}, \dots, x_{i_r}\}$ to be fixed to 1. Then we can forbid assignments x_i , $i < i_r$, i.e., fix $x_i = 0$ for $i < i_r$ if $x_i \notin S$.

2.4.2. The list of assignments L and the enumeration

Suppose that we use the same list of assignments L as [Ulungu and Teghem \(1995\)](#) and that we apply the variable fixing described in Section 2.4.1. In order to be sure to find all efficient solutions it is necessary to enumerate all optimal solutions of every reduced problem. However, for each of the problems with fixed variables we may find many solutions which may be repeated several times because variables with $\bar{c}_{ij}^\lambda = 0$ allow for alternative optima.

We therefore propose a modification of L to include the variables with $\bar{c}_{ij}^\lambda = 0$. This way assignments with reduced cost zero will be imposed (and forbidden) in the combinations of assignments, and no enumeration will be necessary. Although the number of combinations will be larger, numerical evidence suggests that avoidance of redundancy is worth that price, and this alternative is more efficient computationally. Moreover, we will consider solutions one by one which makes it easy to reuse the lower bounds proposed by [Ulungu and Teghem \(1995\)](#) during the depth-first search. This will be shown in the following section.

2.4.3. Stopping conditions

Each time we fix an additional variable to one, the objective value z^λ increases. Say x'' is the new optimal solution of the reduced problem resulting from imposing an assignment in solution x' then $z^\lambda(x'') \geq z^\lambda(x')$. Consequently, if $z^\lambda \geq \beta_0$ we cannot find new efficient solutions by imposing more assignments. This is the first stopping condition.

As [Ulungu and Teghem \(1995\)](#) have observed in the above quote, finding a solution outside of $\Delta(x^r, x^s)$ during a reoptimisation does not mean that a subsequent solution with more imposed assignments will also be outside $\Delta(x^r, x^s)$ again. Therefore finding a solution outside of the triangle is not a stopping condition.

However, even if $z^\lambda(x'') < \beta_0$ not all possible additional variable fixings and reoptimisations are useful. We can use the lower bounds of Section 2.3.1 to test if adding further assignments will give only solutions outside of the triangle or above the upper bound β_0 .

Note that x' is an optimal solution of a reduced problem (BAP_λ) , but not necessarily of the reduced problem with objectives C^1 or C^2 . Let l'_λ, l'_1, l'_2 denote the lower bounds on the objective function values z^λ, z_1 and z_2 , respectively, of a solution resulting from fixing an additional variable. Reoptimisation can be avoided if l'_λ shows that we are sure to find a solution in the hashed area in [Fig. 6](#). If

$$l'_\lambda \geq \beta_0 \quad (1)$$

there is no need to fix the variable.

In the same way, we can use lower bounds on z_1 and z_2 to see if a reoptimisation after variable fixing yields a solution outside of $\Delta(x^r, x^s)$. If one of these two lower bounds shows that this will be the case, all solutions obtained by fixing more variables to 1 will also be outside $\Delta(x^r, x^s)$.

Thus, if

$$l'_1 \geq z_1(x^s), \quad \text{or} \quad (2)$$

$$l'_2 \geq z_2(x^r) \quad (3)$$

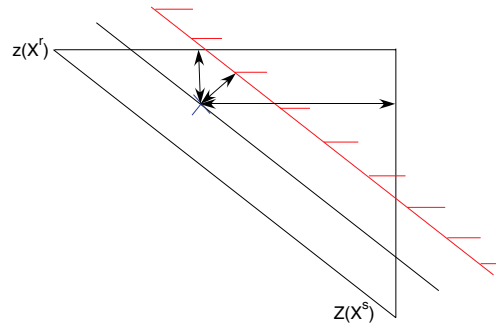


Fig. 6. Test before adding an assignment.

there is no need to fix the variable. So, before fixing a variable, we will check if one of the inequalities (1), (2), or (3) is verified. This is the second stopping condition.

2.4.4. Description of the algorithm

We consider one triangle given by two consecutive supported efficient solutions at a time. If $\lambda_1(z_1(x^s) - 1) + \lambda_2(z_2(x^r) - 1) \leq \lambda_1 z_1(x^r) + \lambda_2 z_2(x^r)$ it is not necessary to continue.

Otherwise we compute the list of assignments L which will be used for the depth-first search: $L := \{(i, j) : \bar{c}_{ij}^{\lambda} \geq 0\}$. Tests 1, 2, and 3 are performed for all of these variables (using `isTest_123` in Algorithm 3). If all pairs with strictly positive reduced cost are deleted by the tests, it is not necessary to continue.

Otherwise the depth-first search is carried out. L is sorted by increasing value of lower bound l_{λ} computed with Test 1. The depth-first search is a recursive function (see procedure `impose` in Algorithms 3 and 4) and fixing variables to 0 is naturally integrated as shown in Section 2.4.1. Before imposing an assignment, compatibility with the other fixed variables is checked (using procedure `compatible` in Algorithm 4).

After each reoptimisation, giving a solution x' , we verify that its value does not violate the upper bound (first stopping condition). In that case, we verify if it is necessary to update the set of potentially efficient solutions X_{PE} (using procedure `isInTriangle`, `isDominated` and `removeDominatedSolution` in Algorithm 4) and the upper bound (using procedure `computeUpperbound` in Algorithm 4). Finally, we check if it is necessary to impose the next compatible assignment by checking the lower bounds (second stopping condition using procedure `NeedtoFixVariable`). If it is, we reiterate the process.

Algorithm 3

Procedure BAP_Phase2

Parameters $\downarrow$: C^1, C^2, X_{SE_m}

Parameters $\uparrow$: S which is X_{NE} or X_{NE_M} depending on the upper bound (β_1 or β_2) selected

$S \leftarrow \emptyset$

for all x^r, x^s consecutive in X_{SE_m} **do**

- | Compute one optimal solution $\tilde{x}$ of (BAP_{λ}) with the cost matrix
- | $C^{\lambda} = [\lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2]$ with $\lambda_1 = z_2(x^r) - z_2(x^s)$ and $\lambda_2 = z_1(x^s) - z_1(x^r)$
- `solveAP` ($C^{\lambda} \downarrow, \tilde{x} \uparrow, \bar{C}^{\lambda} \uparrow$)
- | Let $\bar{C}^{\lambda}$ be a reduced cost matrix of z^{λ} for the problem (BAP_{λ})
- | $\bar{C}^1$ be a reduced cost matrix of z_1 for the solution x^r
- | $\bar{C}^2$ be a reduced cost matrix of z_2 for the solution x^s
- | X_{PE} be the list of potentially efficient solutions of the triangle
- | vUB be the initial value for the upper bound

$L \leftarrow \{(i, j) \mid \bar{c}_{ij}^{\lambda} > 0\}$

$X_{PE} \leftarrow \dots$

–| Empty or initialised with a heuristic (Section 3.2)

```

if  $X_{PE} \neq \emptyset$  then
  computeUpperBound( $X_{PE} \downarrow, vUB \uparrow$ )
else
   $vUB \leftarrow z^i(x^r) + \lambda_1 \lambda_2 - \lambda_1 - \lambda_2$ 
end if
if  $vUB > z^i(x^r)$  then
  –| Test if the assignments give a solution outside of the triangle
  for all  $(i, j) \in L$  do
    if not ( isTest_123( $\bar{C}^i \downarrow, \bar{C}^1 \downarrow, \bar{C}^2 \downarrow, x^r \downarrow, x^s \downarrow, (i, j) \downarrow$ ) ) then
       $L \leftarrow L \setminus \{(i, j)\}$ 
    end if
  end for
  if  $L \neq \emptyset$  then
    –| begin to impose assignments
     $L \leftarrow L \cup \{(i, j) \mid \bar{c}_{ij}^i = 0\}$ 
    for all  $(i, j) \in L$  do
      if NeedtoFixVariable( $\bar{C}^i \downarrow, \bar{C}^1 \downarrow, \bar{C}^2 \downarrow, x^r \downarrow, x^s \downarrow, (i, j) \downarrow$ ) then
        impose( $\{(i, j)\} \downarrow, L \downarrow, vUB \uparrow, X_{PE} \uparrow$ )
      end if
       $\bar{C}^i(x_i) \leftarrow \infty; \bar{C}^1(x_i) \leftarrow \infty; \bar{C}^2(x_i) \leftarrow \infty; C^1(x_i) \leftarrow \infty; C^2(x_i) \leftarrow \infty$ 
    end for
  end if
end if
 $S \leftarrow S \cup X_{PE}$ 
end for

```

Algorithm 4

Procedure impose

Parameters $\downarrow$: $LIST, L$

Parameters $\uparrow$: vUB, X_{PE}

- | Let C^i be the costs matrix of (BAP_i) after the new imposition
- | Let $\bar{C}^i, \bar{C}^1, \bar{C}^2$ be reduced costs matrix after the new imposition
- | Let $LIST$ be the assignments to impose

solveAP($C^i \downarrow, x \uparrow, \bar{C}^i \uparrow$)

if ($z^i(x) \leq vUB$) **then**

if isInTriangle($x^r \downarrow, x^s \downarrow, x \downarrow$) **and not** (isDominated($x \downarrow, X_{PE} \downarrow$)) **then**

$X_{PE} \leftarrow X_{PE} \cup \{x\}$

removeDominatedSolution($X_{PE} \uparrow$)

computeUpperBound($X_{PE} \downarrow, vUB \uparrow$)

end if

–| Let i be the index of the last assignment (pair) in $LIST$

for all $j > i$ according to L **do**

if compatible($x_j \downarrow, LIST \downarrow$) **then**

if NeedtoFixVariable($\bar{C}^i \downarrow, \bar{C}^1 \downarrow, \bar{C}^2 \downarrow, x \downarrow, x_j \downarrow$) **then**

$LIST \leftarrow LIST \cup \{x_j\}$

impose ($LIST \downarrow, L \downarrow, vUB \uparrow, X_{PE} \uparrow$)

$LIST \leftarrow LIST \setminus \{x_j\}$

end if

–| for the following assignments, it is necessary to place x_j at its place in the reduced problem
 $\overline{C}^1 r(x_j) \leftarrow \infty$; $\overline{C}^1 r(x_j) \leftarrow \infty$; $\overline{C}^2 r(x_j) \leftarrow \infty$; $C^1 r(x_j) \leftarrow \infty$; $C^2 r(x_j) \leftarrow \infty$
 end if
 end for
 end if

In the following section, we propose new developments for Phase 2, namely improved upper bounds, the use of a population based heuristic, and a ranking algorithm.

3. Improvements and alternative algorithms for phase 2

3.1. New upper bounds

Like Tuytens et al. (2000), we suppose the objective coefficients to be integer, therefore we can improve the upper bound. Depending on the upper bound different complete sets are obtained.

3.1.1. Improved upper bound

The bound of Tuytens et al. (2000) neglects the fact that $z(x)$ has integer coordinates for all $x \in X$. We can use this fact to improve the upper bound. If we translate the line defined by the bound β_0 towards the supported non-dominated points the first point with integer coordinates encountered is either a point which is one unit down and left of a local nadir point, or one of the potentially non-dominated points (Fig. 7).

Let $\delta_1 = \max_{i=0}^q \{\lambda_1 z_1(x^i) + \lambda_2 z_2(x^i)\}$ be the maximum weighted sum objective for the potential efficient solutions and $\delta_2 = \max_{i=1}^q \{\lambda_1 (z_1(x^i) - 1) + \lambda_2 (z_2(x^{i-1}) - 1)\}$ the maximum for the points one unit down and left of the local nadir points. Then the improved bound is defined by

$$\beta_1 := \max \{\delta_1, \delta_2, \lambda_1 (z_1(x^0) - 1) + \lambda_2 (z_2(x^r) - 1), \lambda_1 (z_1(x^s) - 1) + \lambda_2 (z_2(x^q) - 1)\}.$$

In Fig. 7 the maximum is found for the non-dominated point (91, 59) and has value 2095 (an increase of 40 from x^r and x^s).

All solutions $x \in X, z(x) \in \Delta(x^r, x^s)$ with $\lambda_1 z_1 + \lambda_2 z_2 > \beta_1$ are dominated. Therefore, enumeration of all feasible solutions with $(z_1(x), z_2(x))$ between the lines $\overline{z(x^r)z(x^s)}$ and $\{z : \lambda_1 z_1 + \lambda_2 z_2 = \beta_1\}$ (inclusive) yields all

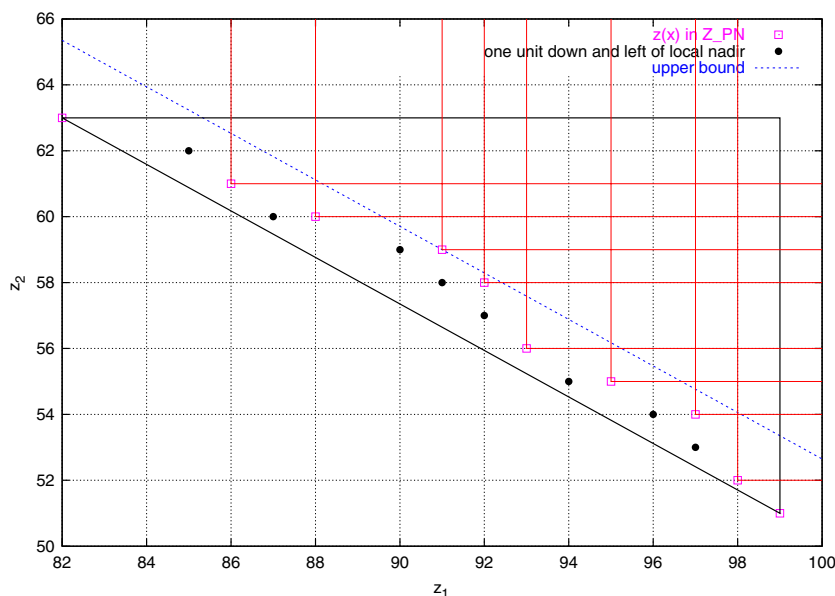
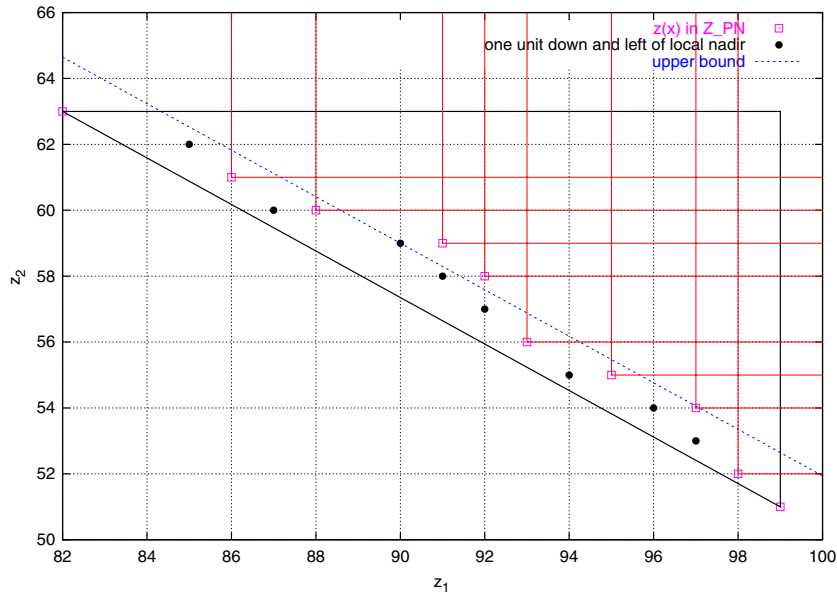


Fig. 7. β_1 : A first improvement of the upper bound of Tuytens et al. (2000).

Fig. 8. Bound β_2 .

non-supported efficient solutions of the triangle including equivalent ones. Thus, after exploring all triangles, we will find the maximal complete set X_{EM} .

3.1.2. An improved bound to find a complete set

If the goal is to find only a minimal complete set, β_1 can be strengthened further, because it is not necessary to consider the potentially efficient solutions when computing β_1 , see Fig. 8.

The bound is defined by

$$\beta_2 := \max \{ \delta_2, \lambda_1(z_1(x^0) - 1) + \lambda_2(z_2(x^r) - 1), \lambda_1(z_1(x^s) - 1) + \lambda_2(z_2(x^q) - 1) \}.$$

Each feasible solution $x \in X, z(x) \in \Delta(x^r, x^s)$ with $\lambda_1 z_1(x) + \lambda_2 z_2(x) > \beta_2$ is dominated or equivalent to one of the potentially efficient solutions. Therefore, enumeration of all feasible solutions with $(z_1(x), z_2(x))$ between the lines $\overline{z(x^r)z(x^s)}$ and $\{z : \lambda_1 z_1 + \lambda_2 z_2 = \beta_2\}$ (inclusive) yields all non-supported efficient solutions in that band. In addition, we already have some potentially efficient solutions outside the band, which may be proven efficient at the end of the exploration. Consequently, after exploring all triangles this way, we will find a complete set X_E , which is at least minimal, but not necessarily maximal.

Further reduction of the bound is not possible, as we could miss some non-dominated points, and consequently keep some dominated ones as non-dominated: β_2 is a tight bound for an exact method with an enumerative exploration strategy.

In Fig. 8 the maximum is found for the point (90, 59) with value 2083 (an increase of 28 from x^r and x^s). That means a complete set of efficient solutions in this triangle is obtained with at most 1.36% increase in the value of z^λ . Recall that $\beta_0 = 2112$ and $\beta_1 = 2095$. Thus using β_2 allows the possible increase in z^λ that needs to be considered for enumeration to be reduced from 57 to 28, i.e., a reduction of 50.88%. This turns out to be very significant, as we shall see in Section 4.2.

In the algorithms, we only use bounds $\beta_i, i = 1, 2$. This implies a slight modification of the stopping conditions. The first stopping condition $z^\lambda \geq \beta_0$ becomes $z^\lambda > \beta_i$ and inequality (1) in the second stopping condition becomes $l'_\lambda > \beta_i$.

3.2. An evolutionary heuristic for computing tight upper bounds

In the Phase 2 algorithm presented in Section 2.4, we start with $X_{PE} = \emptyset$. Therefore the initial upper bound may be bad, and its decrease during the algorithm slow. The idea of the heuristic, called *Seek and Cut*

algorithm, is to search for potentially efficient solutions using a fast heuristic to obtain a good approximation of the non-dominated frontier. These solutions are sorted by triangle, for which the exact Phase 2 algorithm therefore starts with a non-empty X_{PE} and a good upper bound.

We use and modify the heuristic proposed by Gandibleux et al. (in press). It is a population-based heuristic using a path relinking operator. The heuristic uses three operators – local search, crossover, and path relinking – performed on a population of potentially efficient solutions. The initial population is a subset of X_{SE} , e.g., X_{SE1_m} . The heuristic can be used with different parameters to find approximations of X_E of varying quality with respect to CPU time. In particular, the approximation quality is very sensitive to the path relinking operator. We have used three different versions of path relinking.

Path relinking generates new solutions that connect potentially efficient solutions starting from one solution (the initiating solution), generating a path through the solution space using a neighborhood structure that leads toward the other (the guiding) solution (Glover and Laguna, 1997).

A path relinking operator starts by randomly selecting x' and x'' , two individuals in the current potentially efficient population of solutions. Because both are potentially efficient, both can be the guiding solution. Let x' be the initiating solution and x'' the guiding solution. The path relinking operator generates a path $x^0 = x', x^1, \dots, x''$, such that the distance between x^i and x'' decreases along the path, where distance is defined as the number of different positions for tasks assigned in x^i and x'' . At each iteration, a list LDA (list of different assignments) reports the tasks assigned to different positions in solutions x^i and x'' , and the neighbourhood is defined by a simple swap: A new solution is built by selecting one task in LDA, and assigning this task to its final position j in the guiding solution x'' . To maintain a feasible solution, the task currently in position j in x^i is moved to j' , the current position of the selected task. The three variations we consider are as follows:

- *Pure random path relinking* (Gandibleux et al., in press): A new solution is built by randomly selecting one task from LDA. If the solution is a promising solution (i.e., a solution which is not dominated by one of the local nadir points defined by consecutive supported non-dominated points), a local search is performed (see (Gandibleux et al., in press)). We will call this procedure (PR1).
- *Selected random path relinking*. We check each solution which reduces the distance between the initial (or current) and the guiding solution, to check if some are (possibly new) potentially efficient solutions. The set of potentially efficient solutions is updated if necessary. Then we randomly choose one of the potentially

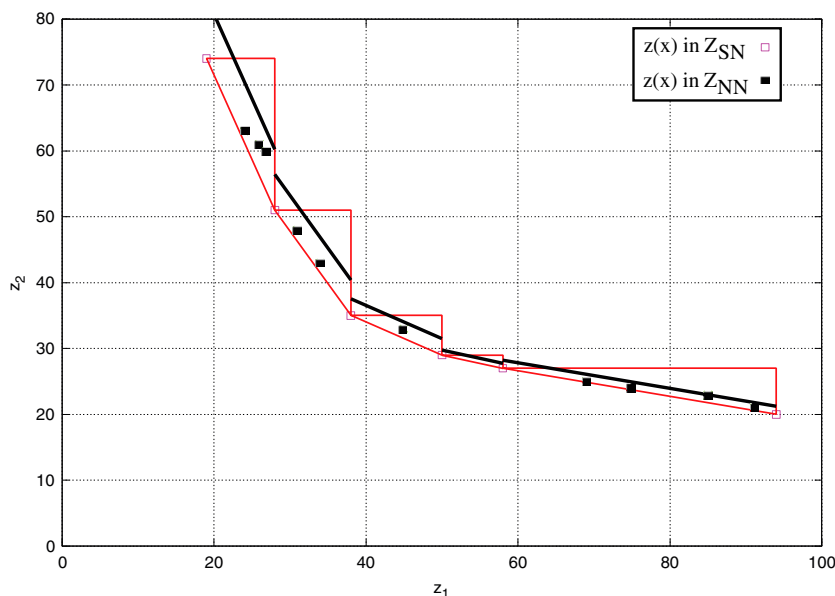


Fig. 9. The exploration space is reduced by the new initial bounds (instance 2AP10-1A20).

efficient solution of the neighborhood of the current solution to continue the path relinking. If there is no potentially efficient solution in the neighborhood the next solution is selected purely randomly. If the new solution is a promising solution, we perform a local search. This procedure is called (PR2).

- *Selected random path relinking with local search.* This method is the same as (PR2), except that for the promising solutions in the neighborhood of the current solution we perform a local search. We will call this (PR3).

The heuristic proposed by Gandibleux et al. (in press) can be used with several stopping conditions. We have tried the following: 250.000 generations for the heuristic (stopping condition (N1)) and stop after 100.000 generations without improvement of X_{PE} (stopping condition (N2)).

Therefore, we have six parameter settings $(PR^*)(N^*)$, some of which are computationally expensive (in particular (PR3) and (N2)). However, the time needed by the heuristic is negligible compared to the time needed by Phase 2 with variable fixing strategy.

We can now describe the algorithm. First, X_{SEM} is determined using Algorithm 1 for Phase 1. The parameters for the heuristic are chosen and the heuristic is run. The set X_{PE} found by the heuristic is sorted by triangle and kept as initial set X_{PE} in Phase 2. We can now consider each triangle separately and start Phase 2. Using the initial set X_{PE} we can compute a good initial upper bound (Fig. 9). From here the Phase 2 proceeds as before.

3.3. A ranking approach for the Phase 2

In this section, we propose a different strategy for the exploration of triangles $\Delta(x^r, x^s)$ defined by consecutive supported efficient solutions x^r, x^s . In Phase 2, we are looking for solutions with increasing values of z^λ until one of the upper bounds β_i is reached. This can be done using a ranking method, i.e., an algorithm that finds solutions in increasing order of their z^λ value.

This procedure is a natural choice in Phase 2. As it does not induce a modification of the problem structure it does not betray the original principle of the two phase method. The main advantages of a ranking algorithm over a variable fixing approach are that redundancy is avoided and that the exploration is ordered, because of the monotonicity with respect to z^λ values.

Algorithm 5

Procedure BAP_Phase2_ranking

Parameters $\downarrow$: C^1, C^2, X_{SEM}

Parameters $\uparrow$: S which is X_{NE} or X_{NEM} according to the upper bound (β_1 or β_2) selected

$S \leftarrow \emptyset$

for all x^r, x^s consecutive in X_{SEM} **do**

–| Compute one optimal solution $\tilde{x}$ of (BAP $_\lambda$) with the cost matrix

–| $C^\lambda = [\lambda_1 c_{ij}^1 + \lambda_2 c_{ij}^2]$ with $\lambda_1 = z_2(x^r) - z_2(x^s)$ and $\lambda_2 = z_1(x^s) - z_1(x^r)$

–| solveAP ($C^\lambda \downarrow, \tilde{x} \uparrow, \bar{C}^\lambda \uparrow$)

–| Let $\bar{C}^\lambda$ be a reduced cost matrix of z^λ for the problem (BAP $_\lambda$)

–| $\bar{C}^1$ be a reduced cost matrix of z_1 for the solution x^r

–| $\bar{C}^2$ be a reduced cost matrix of z_2 for the solution x^s

$L \leftarrow \{(i, j) : \bar{c}_{ij}^\lambda > 0\}$

$R \leftarrow \emptyset$ –| R , the list of efficient solutions in the triangle

$vUB \leftarrow z^\lambda(x^r) + \lambda_1 \lambda_2 - \lambda_1 - \lambda_2$ –| vUB , the initial value for the upper bound

if $vUB > z^\lambda(x^r)$ **then**

–| Test if the assignments give a solution outside of the triangle

for all $(i, j) \in L$ **do**

if not (isTest_123($\bar{C}^\lambda \downarrow, \bar{C}^1 \downarrow, \bar{C}^2 \downarrow, x^r \downarrow, x^s \downarrow, (i, j) \downarrow$))

$c_{ij}^\lambda \leftarrow \infty$

end if

```

end for
if  $L \neq \emptyset$  then
  –| begin the ranking
   $K \leftarrow 0$ 
  while  $(z^i(x^K) \leq vUB)$  do
     $K \leftarrow K + 1$ 
    ComputeNextK_bestSolution( $K \downarrow, C^i \downarrow, x^K \uparrow$ )
    if isInTriangle( $x^r \downarrow, x^s \downarrow, x^K \downarrow$ ) and not (isDominated( $x^K \downarrow, R \downarrow$ )) then
       $R \leftarrow R \cup \{x^K\}$ 
      computeUpperBound( $R \downarrow, vUB \downarrow$ ) –| according to  $\beta_1$  or  $\beta_2$ 
    end if
  end while
end if
if
   $S \leftarrow S \cup R$ 
end if
end for

```

The strategy requires an efficient ranking algorithm. For the assignment problem, such an algorithm has been proposed by [Chegireddy and Hamacher \(1987\)](#). Essentially, this algorithm is an application of the binary search tree algorithm in [Hamacher and Queyranne \(1985\)](#). In the algorithm it is necessary to compute the best and second best solution of a single objective problem. [Chegireddy and Hamacher \(1987\)](#) have shown that the second best solution of a single objective AP is obtained by application of a shortest cycle to the permutation given by the best solution. To obtain this cycle [Chegireddy and Hamacher \(1987\)](#) generate a bi-partite graph on which shortest path algorithms are performed.

We consider the problem (BAP _{λ}) for which consecutive solutions x^r and x^s are optimal and we apply an algorithm to find the k -best solution iteratively. As in the other Phase 2 methods, for each new solution (obtained using procedure ComputeNextK_bestSolution in [Algorithm 5](#)), we verify if it is necessary to update X_{PE} and the upper bound. The procedure stops as soon as we first exceed an upper bound β_i . If we also apply the lower bounds α_i we avoid enumerating points outside the triangle $\Delta(x^r, x^s)$. Due to the ranking approach there is no need to fix variables to one, however, we will forbid variables whose inclusion in a solution leads to points outside the triangle by replacing their cost with a very high number.

While Tests 2 and 3 allow to avoid exploration outside the triangle, Test 1 does not reduce the number of solutions explored. However, the exclusion of assignments obtained with Test 1 implies that the graph generated to compute the second best solution is sparser, and then the ranking algorithm performs slightly faster.

The only stopping condition for this Phase 2 algorithm is given by the upper bound. Note that, as the exploration is naturally ordered, there is no need to remove solutions from X_{PE} . Each new solution given by the ranking, which is not dominated at this stage of the process is definitively efficient.

4. Numerical experiments

4.1. Experimental environment

A library of numerical instances for multi-objective combinatorial optimisation problems is available on the internet at www.terry.uga.edu/mcdm/. The name of an instance provides the following characteristics: the number of objectives, the problem, the size n , the series, the objective type, and the range of objective function coefficients in $[0, value]$. For example, 2AP05-1A20 is a bi-objective (2) assignment problem (AP) with 5×5 variables (05) from series 1 (1); the coefficients of the objective function are generated randomly (A) in the range $[0, 20]$ (20). The coefficients of both objective functions are all generated independently of one another, following a uniform distribution.

The instances 2AP05-1A20 through 2AP50-1A20 have been used in Tuytens et al. (2000). We have completed these instances to obtain a series with size from $n = 5$ to $n = 100$ with one instance for each size. In addition, for sizes $n = 10$ to $n = 100$ with an increment of 10, we have generated ten instances each with objective function coefficients in the range $[0, 20]$ and one instance each with ranges $[0, 40]$, $[0, 60]$, $[0, 80]$, $[0, 100]$.

The computer used for the experiments is equipped with a PowerPC G4 1 GHz processor with 512 MB of RAM, and runs under the OS X operating system. The algorithms have been implemented in C. The binary has been obtained using the compiler gcc with the optimiser option-O3. The exceptions are for a comparison of the two phase method with ranking, using bound β_2 , with CPLEX (on a Pentium 4 3.4 GHz processor with 4 GB RAM under Red Hat Linux) and the results from Tuytens et al. (2000) which we report for comparison (on a DEC 3000 Alpha).

We have used the Hungarian method to solve the single-objective assignment problems (an implementation with the successive shortest path algorithm has given worse results) and the algorithm of Fukuda and Matsui (1992) to find all optimal solutions.

4.2. Analysis

We have analysed the performance of the original two phase method (with our depth-first search implementation of variable fixing), the two phase method with all six variants of the Seek & Cut algorithm, and the two phase method with ranking. In addition, we have tested the time needed to confirm optimality by providing a

Table 1

(1) DEC 3000 alpha, (2) PowerPC G4 1 GHz, 512 MB of RAM, under OS X system, and (3) P4 EE 3, 4 GHz, RAM 4 GB, under Red Hat WS 3 Linux

Instance: n in 2APn-1A20	5	10	15	20	25	30	35	40
$ X_{E_m} $	8	16	39	55	74	88	81	127
$ X_{SE_m} $	3	6	12	13	25	27	27	54
$ X_{NE_m} $	5	10	27	42	49	61	54	73
$ X_{E_m} $	8	20	39	59	92	113	97	217
$ X_{SE_m} $	3	7	12	14	25	30	30	65
$ X_{NE_m} $	5	13	27	45	67	83	67	152
<i>Computer used: (1)</i>								
$ E $ in Tuytens et al. (2000)	8	16	39	54	71	88	92	126
Time in Tuytens et al. (2000)	5	10	14	61	102	183	384	1203
<i>Computer used: (2)</i>								
Two phase β_2	0	0.01	0.16	0.85	3.19	10.66	17.90	60.58
Two phase β_1	0	0.02	0.19	0.93	3.43	12.58	20.72	116.36
Seek & Cut (PR1)(N1) β_2	0.80	1.77	5.96	11.16	10.96	19.80	27.16	45.51
Seek & Cut (PR1)(N2) β_2	0.47	1.04	3.50	6.75	10.71	19.31	26.71	52.95
Seek & Cut (PR2)(N1) β_2	0.86	2.15	7.36	12.89	14.65	26.21	33.99	57.56
Seek & Cut (PR2)(N2) β_2	0.51	1.29	6.08	8.07	9.62	22.15	29.38	58.56
Seek & Cut (PR3)(N1) β_2	1.31	4.19	21.94	44.92	36.50	87.57	90.46	130.04
Seek & Cut (PR1)(N1) β_1	0.78	1.72	5.92	11.26	11.18	21.24	29.29	84.21
Seek & Cut (PR1)(N2) β_1	0.46	1.04	3.56	6.95	11.18	21.16	29.40	92.27
Seek & Cut (PR2)(N1) β_1	0.80	2.12	7.37	12.90	14.89	28.09	36.61	96.84
Seek & Cut (PR2)(N2) β_1	0.49	1.27	5.92	7.91	9.72	23.50	31.51	96.72
Seek & Cut (PR3)(N1) β_1	1.26	4.14	21.88	44.93	36.70	89.29	92.94	168.48
Seek & Cut (PR3)(N2) β_1	0.78	2.50	13.31	27.29	29.98	71.59	59.90	189.99
Seek & Cut X_{NE_m} β_2	0	0	0.10	0.42	1.79	3.40	8.62	24.29
Seek & Cut X_{NE_m} β_1	0	0	0.13	0.51	2.05	5.32	11.21	63.72
Ranking β_2	0	0.01	0.06	0.16	0.49	1.05	1.87	4.04
Ranking β_1	0	0.01	0.06	0.19	0.46	1.01	1.80	4.71
<i>Computer used: (3)</i>								
CPLEX 9.0	0.02	0.14	0.72	1.71	3.13	5.63	7.02	12.21
Ranking β_2	0	0	0.02	0.07	0.18	0.39	0.73	1.55

Table 2

(1) DEC 3000 alpha, (2) PowerPC G4 1 GHz, 512 MB of RAM, under OS X system, and (3) P4 EE 3, 4 GHz, RAM 4 GB, under Red Hat WS 3 Linux

Instance: n in 2APn-1A20	45	50	60	70	80	90	100
$ X_{Em} $	114	163	128	174	195	191	223
$ X_{SEm} $	43	67	44	60	69	83	101
$ X_{NEm} $	71	96	84	114	126	108	122
$ X_{Em} $	170	406	234	413	694	947	1167
$ X_{SEm} $	52	100	53	82	123	236	286
$ X_{NEm} $	118	306	181	331	571	711	881
<i>Computer used: (1)</i>							
$ E $ in Tuytens et al. (2000)	113	156	×	×	×	×	×
Time in Tuytens et al. (2000)	3120	3622	×	×	×	×	×
<i>Computer used: (2)</i>							
Two phase β_2	110.36	270.99	661.57	1980.15	3881.63	10181.73	14049.17
Two phase β_1	170.48	703.09	1416.83	5726.72	13002.79	27916.14	61973.58
Seek & Cut (PR1)(N1) β_2	65.83	101.30	214.50	507.64	1057.23	1721.60	3188.85
Seek & Cut (PR1)(N2) β_2	80.40	113.67	237.88	554.50	1078.08	1650.48	2730.15
Seek & Cut (PR2)(N1) β_2	76.95	118.73	232.08	505.26	946.39	1676.00	2251.72
Seek & Cut (PR2)(N2) β_2	100.98	187.35	366.64	639.35	1208.98	1590.71	2130.31
Seek & Cut (PR3)(N1) β_2	152.57	259.74	325.95	647.44	1188.29	1803.19	2977.79
Seek & Cut (PR1)(N1) β_1	97.22	315.73	516.93	1951.02	4512.73	8395.28	31970.92
Seek & Cut (PR1)(N2) β_1	114.87	331.40	539.33	2032.45	4516.04	7517.66	31565.47
Seek & Cut (PR2)(N1) β_1	111.52	328.14	529.78	2029.92	4447.64	8439.20	31648.29
Seek & Cut (PR2)(N2) β_1	133.30	392.77	619.34	2160.05	4537.33	7459.02	29330.84
Seek & Cut (PR3)(N1) β_1	180.63	469.16	591.29	2064.43	4732.15	7740.68	31266.37
Seek & Cut (PR3)(N2) β_1	180.76	627.50	690.96	2263.56	5543.77	8434.74	31446.25
Seek & Cut X_{NEm} β_2	37.53	57.25	112.17	230.49	528.00	877.60	546.91
Seek & Cut X_{NEm} β_1	68.48	267.10	362.83	1631.07	3857.59	6026.90	26815.00
Ranking β_2	5.40	13.56	11.63	40.32	76.99	120.53	228.26
Ranking β_1	5.71	19.18	13.91	50.86	119.55	201.98	461.36
<i>Computer used: (3)</i>							
CPLEX 9.0	12.45	23.09	36.17	72.20	120.48	135.74	200.63
Ranking β_2	1.99	5.16	4.36	15.02	28.34	44.22	85.58

minimal complete set X_{NEm} as input to Phase 2. Two variants of each method are obtained by using upper bound β_1 or β_2 . Finally, we have used CPLEX as an MIP solver to generate a set X_{Em} . A dichotomic scheme has been designed by Gandibleux et al. (2003). The solver is invoked for problems (BAP $_{\lambda}$) with two additional constraints $z_1(x) < z_1(x^s)$ and $z_2(x) < z_2(x^s)$, where x^r and x^s are two optimal solutions of (BAP $_{\lambda}$).

The results on the first test series are given in Tables 1 and 2. The first six rows give the size of the various efficient sets obtained with all our implementations. The next two rows give the results obtained by the implementation of Tuytens et al. (2000), the following the results of all 17 methods (the setting (PR3)(N2) has not been used for Seek & Cut with upper bound β_2 because the setting (PR3)(N1) is already too computationally expensive). Finally, a row comparing the two phase method with ranking and bound β_2 with CPLEX is given.

The results on the series generated in the ranges $[0, 40]$, $[0, 60]$, $[0, 80]$ and $[0, 100]$ are given in Tables 3 and 4. For a given range, the first six rows give of the various efficient sets obtained with all our implementations. The next three rows give the results of Seek & Cut with the setting (PR2)(N1) and the bound β_2 , and the ranking with the two bounds. Finally, a row comparing the two phase method with ranking and bound β_2 with CPLEX is given.

4.2.1. Results of the two phase method and Seek & Cut algorithm

Whatever the bound, CPU times for the original two phase method increase exponentially with the size of the problem. In addition, the increase in CPU time with upper bound β_1 is larger than with β_2 when problem

Table 3

(1) PowerPC G4 1 GHz, 512 MB of RAM, under OS X system. (2) P4 EE 3, 4 GHz, RAM 4 GB, under Red Hat WS 3 Linux

Size	10	20	30	40	50	60	70	80	90	100
<i>Range 40</i>										
$ X_{E_m} $	21	66	109	186	216	253	331	355	432	429
$ X_{SE_m} $	9	18	28	36	55	62	69	71	85	114
$ X_{NE_m} $	12	48	81	150	161	191	262	284	347	315
$ X_{E_m} $	21	68	119	195	324	323	437	458	711	666
$ X_{SE_m} $	9	18	32	37	62	72	75	74	92	123
$ X_{NE_m} $	12	50	87	158	262	251	362	384	619	543
<i>Computer used: (1)</i>										
S & C (PR2)(N1) β_2	2.79	10.76	26.76	92.19	341.9	716.52	1788.91	2800.62	9131.49	17441.35
Ranking β_2	0.02	0.34	1.57	8.32	19.19	38.85	70.71	124.26	176.88	225.06
Ranking β_1	0.02	0.34	1.58	8.39	19.61	41.43	76.79	129.74	210.19	284.31
<i>Computer used: (2)</i>										
CPLEX 9.0	0.20	3.01	10.91	43.88	68.63	146.13	230.50	328.16	481.50	512.96
Ranking β_2	0	0.13	0.64	3.44	7.26	14.78	26.92	46.55	66.36	83.63
<i>Range 60</i>										
$ X_{E_m} $	17	66	139	259	304	374	460	498	571	585
$ X_{SE_m} $	9	14	30	54	45	58	67	70	85	97
$ X_{NE_m} $	8	52	109	205	259	316	393	428	486	488
$ X_{E_m} $	17	66	148	279	341	425	522	591	696	685
$ X_{SE_m} $	9	14	31	57	47	59	69	70	87	103
$ X_{NE_m} $	8	52	117	222	294	366	453	521	609	582
<i>Computer used: (1)</i>										
S & C (PR2)(N1) β_2	2.30	11.01	29.02	159.69	310.12	1269.92	3146.15	9563.08	20371.83	38553.18
Ranking β_2	0.02	0.49	2.60	10.30	23.44	52.43	100.98	175.00	351.31	399.65
Ranking β_1	0.02	0.55	2.62	10.52	24.03	53.14	102.85	178.85	359.91	415.70
<i>Computer used: (2)</i>										
CPLEX 9.0	0.17	4.23	21.25	79.66	181.17	332.06	520.79	716.82	1172.82	1730.65
Ranking β_2	0	0.20	1.00	4.12	9.06	20.03	38.39	65.64	131.8	149.73

size increases. In fact, at the end of the exploration of a triangle the difference between the objective value of the supported solution and the upper bound is often near zero (for $n > 20$). This implies that in most triangles we need only explore a slim band. Therefore, the improved bound obtained by Seek & Cut before enumeration is effective. This explains the difference in CPU time between the original two phase method and the Seek & Cut algorithm. Note that the computational effort of using the heuristic only shows advantages over the original two phase method for size $n \geq 50$.

Recall that with upper bound β_1 , we obtain the maximum complete set X_{E_m} and with upper bound β_2 , we obtain a complete set. With both bounds, considering a minimal complete set contained in these sets we have obtained the same number of supported efficient solutions, with the same points in objective space Z_N as Degoutin and Gandibleux (2002) with an MIP solver. We can see that the number of equivalent solutions can be large for the largest instances, and that the equivalent solutions are often non-supported.

We also observe that the Seek & Cut method is sensitive to the quality of the non-dominated frontier obtained by the heuristic. For example, in the instance 2AP100-1A20, with bound β_2 , the improvement compared to the two phase method is a factor between 4 or 7 depending on the settings for the heuristic. However, the CPU time for bound β_1 is less sensitive to the results of the heuristic and the gap between the two phase method and Seek & Cut is smaller (a factor 2 for the instance 2AP100-1A20). There is a limit to this, namely, if instead of the heuristic we provide a minimal complete set X_{NE_m} as input. We then test the time used by the two phase method to confirm the results (with bound β_2) or to confirm and find the equivalent solutions (with bound β_1).

The results with bigger range of objective coefficients have given worse results, because the elimination of solutions from enumeration is more effective with a small range of values (resulting in small triangles) than

Table 4

(1) PowerPC G4 1 GHz, 512 MB of RAM, under OS X system. (2) P4 EE 3,4 GHz, RAM 4 GB, under Red Hat WS 3 Linux

Size	10	20	30	40	50	60	70	80	90	100
<i>Range 80</i>										
$ X_{Em} $	25	94	158	218	375	431	477	677	691	845
$ X_{SEm} $	8	23	23	36	48	56	66	79	101	102
$ X_{NEm} $	17	71	135	182	327	375	411	598	590	743
$ X_{Em} $	25	95	161	250	396	458	513	773	759	1050
$ X_{SEm} $	8	23	23	40	48	56	66	79	101	106
$ X_{NEm} $	17	72	138	210	348	402	447	694	658	944
<i>Computer used: (1)</i>										
S & C (PR2)(N1) β_2	3.11	12.28	31.70	151.11	528.00	1430.81	3644.27	13224.91	20411.17	53747.45
Ranking β_2	0.02	0.52	2.59	10.63	29.60	64.16	118.03	278.04	329.27	721.08
Ranking β_1	0.02	0.54	2.67	10.67	29.74	64.88	120.33	282.33	338.74	774.85
<i>Computer used: (2)</i>										
CPLEX 9.0	0.24	5.45	34.31	79.33	271.04	630.10	851.87	1631.91	1918.06	3766.00
Ranking β_2	0	0.20	1.07	4.30	11.5	24.73	44.93	104.27	124.44	274.06
<i>Range 100</i>										
$ X_{Em} $	13	82	169	243	301	470	573	671	722	947
$ X_{SEm} $	7	17	25	35	33	55	69	77	83	85
$ X_{NEm} $	6	65	144	208	268	415	504	594	639	859
$ X_{Em} $	13	83	169	247	312	485	641	726	774	1046
$ X_{SEm} $	7	17	25	36	34	55	73	77	84	89
$ X_{NEm} $	6	66	144	211	278	430	568	649	690	957
<i>Computer used: (1)</i>										
S & C (PR2)(N1) β_2	1.99	13.71	39.00	150.76	345.17	2455.20	4433.13	13209.98	26378.03	60227.31
Ranking β_2	0.01	0.25	3.52	10.37	21.68	98.76	144.37	257.59	370.09	711.97
Ranking β_1	0.01	0.29	3.59	10.51	21.81	100.34	147.35	263.81	380.96	739.32
<i>Computer used: (2)</i>										
CPLEX 9.0	0.10	5.28	49.17	131.16	295.77	786.79	1252.00	1883.22	2732.00	4822.00
Ranking β_2	0	0.11	1.45	4.20	8.56	38.24	55.67	99.11	142.22	275.09

with a big range, when the triangles are larger. The difference between the upper bounds β_1 and β_2 relative to the objective values is smaller in the case of a bigger range.

4.2.2. Results of the method using a ranking algorithm

We have done some more tests on the method using ranking on the series of 10 instances of size 10, ..., 100 with range [0, 20]. These results are reported in Table 5. Here, we report the minimum, maximum and average time needed to solve the 10 instances of each size for Seek & Cut with parameters (PR2)(N1) and bound β_2 and ranking with both β_2 and β_1 .

The CPU times obtained with the ranking method are better by a factor 10–100. Moreover, this method is less sensitive to the bound used and the range. This can be explained by the fact that no solution is repeated and we never consider solutions which exceed the final bound of the exploration. The distribution of the solutions in the objective space also provides justification that the ranking method is a better approach for solving the (BAP) (see Section 4.2.3 below). The two phase method with ranking has been compared with CPLEX and it seems that one important advantage of the method is that it is robust with respect to the range of objective function coefficients.

Note that we have not tried to improve the results of the ranking method by using a heuristic to improve the initial bound because the time needed by the heuristic and by the ranking method is not very different.

4.2.3. Distribution of objective values and its consequences

The huge improvement in CPU time obtained by the ranking methods can be explained by the huge difference in the number of enumerated solutions as compared to the original two phase Method (with or without

Table 5
CPU time in seconds for instances with range 20

Size	Seek & Cut (PR2)(N1) β_2			Ranking β_2			Ranking β_1		
	Min	Max	Mean	Min	Max	Mean	Min	Max	Mean
10	1.52	2.93	2.26	0	0.02	0.01	0	0.02	0.01
20	6.75	15.52	10.33	0.08	0.25	0.16	0.08	0.26	0.18
30	20.75	29.96	23.88	0.80	1.91	1.10	0.89	2.02	1.19
40	45.28	69.74	57.15	2.23	5.73	3.40	2.46	6.98	4.44
50	92.89	155.05	138.70	5.57	11.14	9.22	5.90	12.53	10.50
60	262.68	379.11	304.76	14.83	32.71	21.68	18.07	49.19	28.34
70	394.94	749.68	521.76	25.36	46.4	34.76	34.71	58.13	46.40
80	627.78	1139.57	964.80	31.20	85.11	54.19	42.7	199.89	86.18
90	1201.10	2325.85	1614.93	54.32	112.36	85.18	85.32	158.23	125.15
100	1711.96	4049.85	2755.75	115.55	194.08	137.18	179.11	290.42	220.07

heuristic). We have observed that near the boundary of the convex hull of Z there are relatively few feasible points. The number of feasible points increases exponentially with increasing objective values. Indeed, we can conjecture that the distribution of objective function values for a large size follows a normal distribution.

Experimentally, we have enumerated the objective function values of all feasible solutions of an AP with $n = 10$. The distribution matches a normal distribution closely. In fact, the mean and variance of a normal distribution fitted to the experimental distribution matches those predicted by the central limit theorem exactly, see Fig. 10.

Consequently, for large n , the distribution of a bi-objective assignment problem empirically follows a two-dimensional normal distribution. Under this observation the supported non-dominated points are on the lower left boundary of the two-dimensional normal density and the non-supported non-dominated points are near the hypotenuse defined by two consecutive supported non-dominated points. This explains why the final upper bounds in the exploration of the triangles were so small and it also explains the gap between the two proposed methods. With the ranking approach we only enumerate solutions which are located near the boundary of the triangles, and therefore at the lower end of the distribution. Any slight increase in values encountered during enumeration can lead to a very large increase in the number of solutions enumerated. This is why the theoretically minor improvement of β_2 over β_0 is significant in practice.

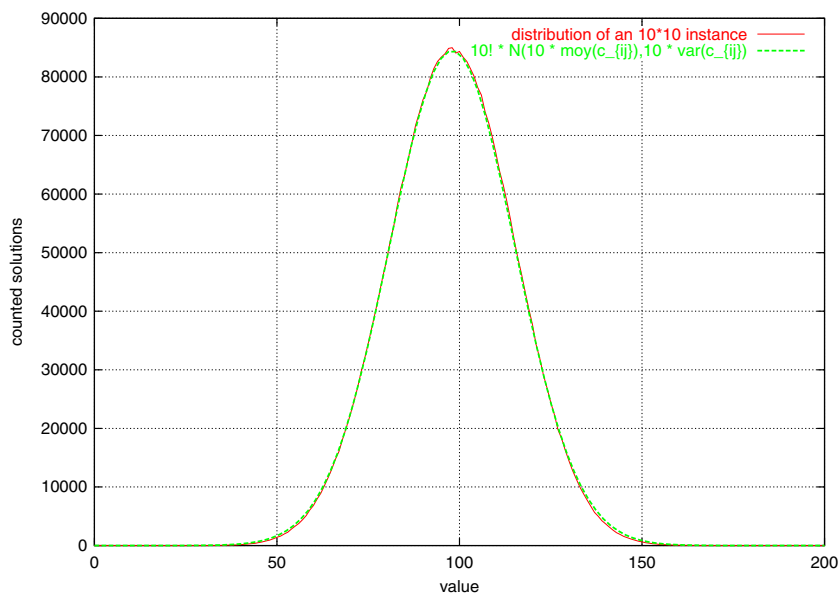


Fig. 10. Counting solutions of an assignment problem obtained by enumeration.

Another consequence of this distribution is the number of efficient solutions. With a small range, the triangles defined by the supported non-dominated points are often small, and because the other non-dominated points are located near the hypotenuse of the triangle, there are fewer non-supported non-dominated points for a small range and more for a big range. However, the variance of objective values increases with increasing range of c_{ij}^k , so the number of equivalent solutions decreases. And since the upper bound is an integer, the gap between the upper bound and the value of z^k for the supported solutions remains the same, so the number of solutions inside this small band is larger. Consequently, the number of equivalent solutions with a very small range is very large and this kind of instance can be difficult to solve. In practice, with range 20, the number of equivalent supported solutions is still significant.

5. Conclusions and future research

In this paper, we have described the original two phase method (with some modifications). We proposed new upper bounds and a strategy of variable fixing with the goal of reducing computational effort of Phase 2. We also proposed the alternative algorithmic approaches of Seek & Cut and ranking for Phase 2. We conclude that the use of a ranking algorithm in Phase 2 implies a huge improvement of the performance of the method in comparison to the original Phase 2, even with the discussed modifications. This conclusion is validated by our experiments and by our observation of the distribution of objective values. We designed a method which outperforms all existing methods and which is very robust with respect to the range of objective coefficients. Our new goals are now to generalise the two phase method to three and more objectives and to extend our observations on the objective values to other kinds of instances.

References

- Ahuja, R.K., Maganti, T.L., Orlin, J.B., 1993. *Network Flows – Theory, Algorithms, and Applications*. Prentice-Hall.
- Aneja, Y.P., Nair, K.P.K., 1979. Bicriteria transportation problem. *Management Science* 25, 73–78.
- Chegireddy, C.R., Hamacher, H.W., 1987. Algorithms for finding k -best perfect matchings. *Discrete Applied Mathematics* 18, 155–165.
- Degoutin, F., Gandibleux, X., 2002. Un retour d'expérience sur la résolution de problèmes combinatoires bi-objectifs. 5e journée du groupe de travail Programmation Mathématique MultiObjectif (PM2O), Angers, France, 2002.
- Ehrgott, M., 2005. *Multicriteria Optimization*, second ed. Springer Science & Business Media.
- Ehrgott, M., Gandibleux, X., 2002. Multiobjective combinatorial optimization. In: Ehrgott, M., Gandibleux, X. (Eds.), *Multiple Criteria Optimization: State of the Art Annotated Bibliographic Surveys*, Kluwer's International Series in Operations Research and Management Science, vol. 52. pp. 369–444.
- Ehrgott, M., Ryan, D.M., 2003. The method of elastic constraints for multiobjective combinatorial optimization and its application in airline crew scheduling. In: Tanino, T., Tanaka, T., Inuiguchi, M. (Eds.), *Multi-Objective Programming and Goal-Programming*. Springer-Verlag, pp. 117–122.
- Fukuda, K., Matsui, T., 1992. Finding all the perfect matchings in bipartite graphs. *Networks* 22, 461–468.
- Gandibleux, X., Morita, H., Katoh, N., 2003. Use of a genetic heritage for solving the assignment problem with two objectives. In: Fonseca, C., Fleming, P., Zitzler, E., Deb, K., Thiele, L. (Eds.), *Evolutionary Multi-Criterion Optimization*, Lecture Notes in Computer Sciences, vol. 2632. Springer, pp. 43–57.
- Gandibleux, X., Morita, H., Katoh, N., in press. A population-based metaheuristic for solving assignment problems with two objectives. *Journal of Mathematical Modelling and Algorithms*.
- Geoffrion, A.M., 1968. Proper efficiency the theory of vector maximization. *Journal of Mathematical Analysis and Applications* 22, 618–630.
- Glover, F., Laguna, M., 1997. *Tabu Search*. Kluwer Academic Publishers.
- Hamacher, H.W., Queyranne, M., 1985. k -Best solutions to combinatorial optimization problems. *Annals of Operations Research* 4, 123–143.
- Hansen, P., 1979. Bicriterion path problems. In: Fandel, G., Gal, T. (Eds.), *Multiple Criteria Decision Making Theory and Application*, Lecture Notes in Economics and Mathematical Systems, vol. 177. Springer Verlag, pp. 109–127.
- Lee, H., Pulat, P.S., 1993. Bicriteria network flow problems: Integer case. *European Journal of Operational Research* 66, 148–157.
- Malhotra, R., Bhatia, H.L., Puri, M.C., 1982. Bi-criteria assignment problem. *Opsearch. Journal of the Operational Research Society of India* 19 (2), 84–96.
- Mavrotas, G., Diakoulaki, D., 1998. A branch and bound algorithm for mixed zero-one multiple objective linear programming. *European Journal of Operational Research* 107 (3), 530–541.
- Mazzola, J.B., Neebe, A.W., 1986. Resource-constrained assignment scheduling. *Operations Research* 34 (4), 560–572.

- Neumayer, P., 1994. Complexity of optimization on vectorweighted graphs. In: Bachem, A., Derigs, U., Jünger, M., Schrader, R. (Eds.), *Operations Research* 93. Physica Verlag, Heidelberg, pp. 359–361.
- Papadimitriou, C.H., Steiglitz, K., 1982. *Combinatorial Optimization*. Prentice Hall, Englewood Cliffs.
- Ramos, R.M., Alonso, S., Sicilia, J., González, C., 1998. The problem of the optimal biobjective spanning tree. *European Journal of Operational Research* 111, 617–628.
- Sedeño-Noda, A., González-Martín, C., 2001. An algorithm for the biobjective integer minimum cost flow problem. *Computers & Operations Research* 28 (2), 139–156.
- Serafini, P., 1986. Some considerations about computational complexity for multiobjective combinatorial problems. In: Jahn, J., Krabs, W. (Eds.), *Recent Advances and Historical Development of Vector Optimization*, Lecture notes in Economics and Mathematical Systems, vol. 294. Springer Verlag.
- Steuer, R.E., 1985. *Multiple Criteria Optimization: Theory, Computation and Application*. John Wiley & Sons.
- Tuytens, D., Teghem, J., Fortemps, Ph., Van Nieuwenhuyse, K., 2000. Performance of the MOSA method for the bicriteria assignment problem. *Journal of Heuristics* 6, 295–310.
- Ulungu, E.L., 1993. *Optimisation Combinatoire multicritère: Détermination de l'ensemble des solutions efficaces et méthodes interactives*. PhD thesis, Université de Mons-Hainault, Faculté des Sciences, 1993.
- Ulungu, E.L., Teghem, J., 1995. The two phases method: An efficient procedure to solve bi-objective combinatorial optimization problems. *Foundations of Computing and Decision Sciences* 20, 149–165.
- Visée, M., Teghem, J., Pirlot, M., Ulungu, E.L., 1998. Two-phases method and branch and bound procedures to solve the bi-objective knapsack problem. *Journal of Global Optimization* 12, 139–155.